CSC2103 Data Structures | Lecture 4

Binary Heaps and Priority Queues

School of Computing (demo) | Week 4

Sample lecture deck compiled from Wikipedia (CC BY-SA 4.0) for a demo.
Not official material of any university. See LICENSE.md for sources and attribution.
Wikipedia articles used: Priority queue, Binary heap, Heapsort.

Learning outcomes

- Describe the priority queue ADT and where it is used
- Store a binary heap in an array and navigate it with index formulas
- Trace insert (sift-up) and extract-min (sift-down)
- Explain why bottom-up build-heap is O(n), and how heapsort works

Priority queues

- Each element has a **priority**; the element with the highest priority leaves first
- Operations: **insert**, **peek** (find-min or find-max), **extract-min** (or extract-max)
- Applications:
  - CPU and job scheduling
  - Dijkstra's shortest paths and Prim's minimum spanning tree
  - Event-driven simulation
  - Heapsort, and finding the k largest items in a stream

Binary heap

- Introduced by J. W. J. Williams in 1964 as the data structure for heapsort
- **Shape property**: a **complete** binary tree: every level full except possibly the last, which fills from the left
- **Heap property**:
  - **min-heap**: every parent ≤ its children, so the minimum is at the root
  - **max-heap**: every parent ≥ its children, so the maximum is at the root
- A heap is **not** sorted: siblings have no order between them

Array representation

- Store the nodes level by level in an array (index 0 = root)
- For the node at index i:
  - left child: **2i + 1**
  - right child: **2i + 2**
  - parent: **⌊(i − 1) / 2⌋**
- No pointers needed; the complete shape leaves no gaps

```
array: [1, 3, 6, 5, 9, 8]

          1
        /   \
       3     6
      / \   /
     5   9 8
```

Insert: sift-up

```
function insert(heap, x) {
  heap.push(x);                          // keep the shape: add at the end
  let i = heap.length - 1;
  while (i > 0) {
    const p = Math.floor((i - 1) / 2);
    if (heap[p] <= heap[i]) break;       // heap property holds
    [heap[p], heap[i]] = [heap[i], heap[p]];
    i = p;                               // move up one level
  }
}
```

- At most one swap per level: **O(log n)** worst case (O(1) on average)

Insert walkthrough

- Start: [1, 3, 6, 5, 9, 8]; insert **2**
- Place 2 at index 6; its parent is index 2 (value 6)
- 2 < 6: swap → [1, 3, **2**, 5, 9, 8, 6]
- Now at index 2; parent is index 0 (value 1); 2 > 1: stop
- Result: **[1, 3, 2, 5, 9, 8, 6]**

Extract-min: sift-down

```
function extractMin(heap) {
  const min = heap[0];
  const last = heap.pop();
  if (heap.length > 0) {
    heap[0] = last;                        // move the last leaf to the root
    let i = 0;
    while (true) {
      const l = 2 * i + 1, r = l + 1;
      let s = i;                           // index of the smallest of i, l, r
      if (l < heap.length && heap[l] < heap[s]) s = l;
      if (r < heap.length && heap[r] < heap[s]) s = r;
      if (s === i) break;
      [heap[s], heap[i]] = [heap[i], heap[s]];
      i = s;
    }
  }
  return min;
}
```

Extract-min walkthrough

- Start: [1, 3, 2, 5, 9, 8, 6]; remove the root **1**
- Move the last element (6) to the root → [6, 3, 2, 5, 9, 8]
- Children of index 0 are 3 and 2; the smaller is 2: swap → [2, 3, **6**, 5, 9, 8]
- Index 2 has one child, 8; 6 < 8: stop
- Result: **[2, 3, 6, 5, 9, 8]**, cost **O(log n)**

Build-heap in O(n)

- Bottom-up (Floyd, 1964): for i from $\lfloor n/2 \rfloor - 1$ down to 0, sift-down(i)
- Leaves are already heaps; most nodes are near the bottom and sift down only a little
- The sum of all node heights is O(n), so build-heap is **O(n)**
- Inserting n keys one at a time is O(n log n) in the worst case
- Example (min-heap): [9, 4, 7, 1, 2, 6] → [9, 4, 6, 1, 2, 7] → [9, 1, 6, 4, 2, 7] → **[1, 2, 6, 4, 9, 7]**

Heapsort

- 1. Build a **max-heap** from the array: O(n)
- 2. Repeat n − 1 times: swap the root with the last element of the heap, shrink the heap by one, sift-down the new root
- Runs in **O(n log n)** in every case
- Sorts **in place** (O(1) extra memory) but is **not stable**

Complexity summary

Operation	Binary heap	Unsorted array	Sorted array
find-min	O(1)	O(n)	O(1)
insert	O(log n)	O(1)	O(n)
extract-min	O(log n)	O(n)	O(1)*
decrease-key	O(log n)	O(1)	O(n)
build from n items	O(n)	O(1)	O(n log n)

* when the array is sorted in decreasing order so the minimum is removed from the end.

Summary and tutorial

- Heap = complete binary tree + heap property, stored in an array
- insert and extract-min are O(log n); find-min is O(1); build-heap is O(n)
- Heapsort: O(n log n), in place, not stable

Tutorial 4: (1) Insert 20, 15, 30, 5, 10, 25 into an empty min-heap and show the array after each insert. (2) Perform two extract-min operations on the result. (3) Use a min-heap of size k to find the k largest numbers in a stream of n numbers. What is the running time?

References and licence

- Wikipedia, *Priority queue*, https://en.wikipedia.org/wiki/Priority_queue (accessed 10 Oct 2026)
- Wikipedia, *Binary heap*, https://en.wikipedia.org/wiki/Binary_heap (accessed 10 Oct 2026)
- Wikipedia, *Heapsort*, https://en.wikipedia.org/wiki/Heapsort (accessed 10 Oct 2026)

Text adapted and condensed from the articles above, licensed CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0/). This deck is shared under the same licence. Code snippets written for this deck.